온리티칭 확장 전략 — 보강 + 회원 50만명 대비 단계별 가이드

작성일 2026-04-08 · ERD 보강 + 1만~50만 사용자 단계별 인프라 전략

1단계 — 즉시 보강 (P0~P2 매핑 보완)

P0 AUTOBIO_JOBS.sourceData 통째 스냅샷

문제: 자서전 생성 시 다른 테이블 JOIN하면 사용자 늘수록 기하급수적으로 느려짐.

해결: sourceData JSON에 필요한 모든 데이터를 미리 복사해서 저장

```
{
  "careRecords": [...],     // 돌봄교실 전체 (이미 있음)
  "students": [...],        // 학생 명단 스냅샷 (이름·번호)
  "schedules": [...],       // 학사일정 스냅샷
  "lifeRecords": [...],     // 생활기록부
  "snapshotAt": "2026-04-08T09:30:00Z"
}
```

→ 자서전 재생성 시 JOIN 0회, 학생 전학 가도 기록 그대로

P1 meal_likes 조인 테이블

```
CREATE TABLE meal_likes (
  likeId       BIGINT PK AUTO_INCREMENT,
  mealId       BIGINT FK,
  studentId    BIGINT FK,
  createdDate  TIMESTAMP,
  status       VARCHAR(20) DEFAULT 'NORMAL',
  UNIQUE(mealId, studentId),
  INDEX(studentId, createdDate DESC)
);
```

→ 누가 좋아요 눌렀는지, 학생별 선호도 분석 가능

P2 creative_activity_students 조인 테이블

```
CREATE TABLE creative_activity_students (
  id            BIGINT PK,
  activityId    BIGINT FK,
  studentId     BIGINT FK,
  participation VARCHAR(20),  -- '참여', '리더', '결석'
  reflection    TEXT,
  UNIQUE(activityId, studentId),
  INDEX(studentId)
);
```

→ 학생별 창체활동 내역을 자서전에 사용 가능

2단계 — 회원 1만명 → 10만명 (10배 확장)

가장 먼저 터지는 곳 3가지

병목	증상	해결
DB CPU 100%	모든 API 1초+	Read Replica
자서전 생성 줄서기	동시 100명 → 서버 다운	비동기 큐
로그인 폭주 (오전 8시)	5000명 동시 접속 → 멈춤	Redis 세션 + Edge 함수

2-1. Read Replica 도입 (필수)

```
[App] —쓰기→ [Primary DB]
              ↓ replication
```

```
        [Read Replica × 2]
              ↑ 읽기 90%
```

- 읽기 전용 쿼리는 전부 Replica로: 자서전 생성, 학사일정, 학생 명부
- Vercel Postgres 자동 지원, Neon Pro도 OK
- 비용: 월 $50~100, 사용자 10만명이면 무조건 값어치

2-2. Redis 캐싱 (필수)

캐시 대상	TTL	효과
학사일정 (학교별)	1일	매일 1번만 DB
학생 명부 (반별)	1시간	자서전 생성 즉시
자서전 결과 (입력 해시)	7일	같은 자서전 0초
사용자 세션	30일	로그인 0 DB 호출
성취기준 마스터	7일	정적 데이터

→ DB 부하 60~80% 감소

2-3. 자서전 비동기 큐 (BullMQ)

```
사용자 클릭
   ↓
API: jobId 즉시 반환 (0.1초)
   ↓
Redis 큐 enqueue
   ↓
워커 5개가 동시 처리
   ↓
완료되면 SSE/Pusher로 알림
```

2-4. 인덱스 강화

```
CREATE INDEX idx_autobio_status_created ON autobio_jobs(jobStatus, createdDate);
CREATE INDEX idx_mood_student_date ON care_student_moods(studentId, createdDate DESC);
CREATE INDEX idx_schedule_major ON schedules(schoolId, isMajorEvent, startDate)
  WHERE isMajorEvent = 1;
```

2-5. 페이지네이션 — 커서 기반

```
❌ OFFSET (페이지 깊을수록 느려짐)
SELECT * FROM care_records WHERE teacherId = ?
  ORDER BY recordDate DESC LIMIT 30 OFFSET 5000;

✅ Cursor (항상 일정한 속도)
SELECT * FROM care_records
  WHERE teacherId = ? AND recordDate < '2026-01-15'
  ORDER BY recordDate DESC LIMIT 30;
```

3단계 — 10만명 → 50만명 (5배 더 확장)

새로 터지는 곳

병목	증상	해결
단일 DB row 수 폭증	care_records 1억건+	파티셔닝
로그 테이블 무한 증식	tool_logs, observation_logs	아카이빙
이미지/파일 DB 점유	DB 용량 폭발	S3/R2
단일 지역 latency	해외 사용자 느림	CDN + Edge
알림·실시간 이벤트	WebSocket 폭주	Pub/Sub 분리

3-1. 테이블 파티셔닝 (시간 기반)

```
CREATE TABLE care_records (
  ...
) PARTITION BY RANGE (YEAR(recordDate)) (
  PARTITION p2024 VALUES LESS THAN (2025),
  PARTITION p2025 VALUES LESS THAN (2026),
  PARTITION p2026 VALUES LESS THAN (2027),
  PARTITION p_future VALUES LESS THAN MAXVALUE
);
```

→ 1억 건 → 5천만 × 2개 → 쿼리 50% 빨라짐

3-2. 콜드 데이터 아카이빙

```
3년 이상 지난 데이터:
  ├─ Hot 테이블에서 분리
  ├─ archive_care_records 테이블로 이동
  └─ 별도 DB 인스턴스로 격리
```

3-3. 파일 외부 스토리지 (필수)

항목	저장소
학생 사진	Cloudflare R2 (egress 무료)
급식 사진	Cloudflare R2
자서전 PDF	Cloudflare R2 + signed URL
첨부파일	Cloudflare R2

DB에는 URL만 저장.

3-4. CDN + Edge Functions

```
[Cloudflare CDN]
   ↓ 정적 자산 (95% 캐시 히트)
[Vercel Edge Functions]  ← 로그인, 학사일정 조회 (이미 적용)
   ↓
[App Server] ← 무거운 API만
   ↓
[Primary DB + Replicas]
```

한국 외 사용자도 100ms 이내 응답

3-5. 도메인 샤딩 (학교 단위)

```
shard_1: 학교 1~1000 (서울/경기)
shard_2: 학교 1001~2000 (충청)
shard_3: 학교 2001~3000 (경상)
...
```

학교 코드로 라우팅 → 한 학교 데이터는 한 샤드에만

3-6. 도메인별 마이크로서비스 분리 (선택)

```
auth-service           (로그인 - 가장 중요)
care-service           (돌봄교실)
records-service        (생활기록부·관찰일지)
autobiography-service  (자서전 - GPU/AI 인스턴스)
schedule-service       (학사일정)
```

3-7. 메시지 큐 + 이벤트 소싱

```
사용자 액션
   ↓
Kafka / Redis Stream
   ↓ (이벤트 발행)
구독자들이 비동기 처리
   ├─ 통계 집계
```

```
├─ 자서전 워커
├─ 알림 발송
└─ 검색 인덱싱
```

4단계 — 사용자 규모별 비용·작업 로드맵

규모	핵심 작업	월 인프라 (대략)	작업 기간
~1,000명	인덱스만 추가	$20 (Vercel Hobby)	1일
~10,000명	캐싱, 큐, 인덱스	$100~200	1주
~100,000명	Replica + Redis + 큐 + 인덱스	$500~1,000	2주
~500,000명	파티셔닝 + 아카이빙 + R2 + CDN + 샤딩	$2,000~5,000	1달
~1,000,000명	마이크로서비스 + 이벤트 소싱	$10,000+	2~3달

한 줄 요약

단계	핵심 기술	효과
즉시	sourceData 스냅샷, 조인 테이블 2개	매핑 100점
1만명	인덱스 6개 + 페이지네이션	응답 100배
10만명	Read Replica + Redis + BullMQ	DB 부하 80% ↓
50만명	파티셔닝 + S3 + CDN + 샤딩	무한 확장 가능